

JavaExpert-Qwen2.5-3B

Domain-Locked Java QA via QLoRA on Consumer Hardware

Author: Debarun Das

Date: June 2026

Hardware: NVIDIA GeForce RTX 5050 · 8 GB VRAM

Model: huggingface.co/Debarun12/JavaExpert-Qwen2.5-3B

EXECUTIVE SUMMARY

Most LLM fine-tuning work happens on cloud A100s with unlimited VRAM budgets. This project did it differently: a production-ready Java programming assistant, built on a single 8 GB consumer GPU, that answers Java questions accurately and refuses everything else — no cloud, no shortcuts.

The core engineering challenge was not just fitting training into 8 GB, but doing so without sacrificing model quality. Every configuration decision — LoRA rank, target modules, optimizer choice, batch strategy — was driven by explicit VRAM accounting. The result is a merged, inference-ready model that runs on under 2 GB VRAM, achieves validation perplexity of 2.40 against a target of <10, and passes manual domain restriction testing with an 8.5/10 refusal score.

Metric	Target	Achieved
Validation Perplexity	< 10	2.40 ✓
Peak Training VRAM	≤ 8 GB	< 7 GB ✓
Inference VRAM	—	< 2 GB
Java Correctness	—	8.5 / 10
Domain Restriction	—	8.5 / 10
Hallucination Control	—	8.0 / 10

1. PROBLEM STATEMENT

General-purpose LLMs fail in two predictable ways in domain-restricted deployments: they hallucinate confidently on unfamiliar material, and they answer questions they shouldn't. This project treats both as first-class engineering problems, not afterthoughts.

The objective was a domain-locked Java assistant satisfying four hard constraints:

- Fits within 8 GB VRAM — no cloud dependency, deployable on commodity hardware
- Validation perplexity <10 on held-out Java QA data
- Refuses non-Java queries without external guardrails or post-processing filters
- Ships as a single merged model — no adapter loading, no runtime dependencies

2. MODEL SELECTION

Base model selection required navigating a three-way trade-off between Java capability, hallucination tendency, and VRAM footprint after 4-bit quantization.

Model	VRAM (4-bit)	Java Knowledge	Hallucination Risk	Decision
Qwen2.5-7B	~4.2 GB	Very High	Low	✗ Rejected (OOM risk)
Qwen2.5-3B	~1.9 GB	High	Low	✓ Selected
Qwen2.5-1.5B	~1.1 GB	Medium	Medium-High	✗ Rejected

The 7B model was rejected not because it couldn't fit in 4-bit, but because the remaining VRAM headroom was insufficient for adapter weights, optimizer states, and gradient buffers at any usable batch size. The 1.5B model's higher hallucination tendency made it a poor foundation for a domain-restriction use case. The 3B sits at the right point on the capability-memory curve.

3. DATA ENGINEERING PIPELINE

3.1 Corpus Preprocessing

The raw corpus — 42,000+ lines of Java documentation extracted from PDFs — contained OCR artifacts, table-of-contents sections, degenerate short chunks, and structural noise from PDF layout parsing. A custom preprocessing pipeline was built from scratch rather than relying on off-the-shelf chunkers.

Chunking: Sliding window with size 150 words and overlap 30 words, preserving cross-sentence context at chunk boundaries.

Filtering — each chunk rejected on any of:

- Mean word length > 12 characters — catches PDF text smashing and OCR garbage
- Absence of Java-specific keywords — removes generic prose with no technical content
- Chunk length < 30 words — eliminates outline stubs and section headers
- > 20 consecutive dots — removes table-of-contents dot leaders
- Chapter/section marker patterns — removes structural navigation text

3.2 LLM-Assisted QA Generation

Valid chunks were passed to qwen2.5:7b via ollama to generate four QA pairs per chunk across four question archetypes — definitional (what), procedural (how), reasoning (why), and out-of-domain refusal (unanswerable). The unanswerable category is critical: it provides the negative training signal that teaches the model when not to answer, not just how to answer.

The generation pipeline required two rounds of prompt engineering to eliminate systematic failure modes: verbatim chunk copying and response truncation on longer explanations.

3.3 Final Dataset

Property	Value
Total QA pairs	7,921

Property	Value
Question distribution	36% what / 32% how / 32% why
Estimated cleanliness	95%+
Format	Chat-style JSONL (system + user + assistant turns)

4. FINE-TUNING STRATEGY

4.1 Why QLoRA

Full fine-tuning of a 3B model in fp16 requires approximately 14 GB VRAM — nearly double the available budget. Standard LoRA keeps the base model in fp16 and still hits the ceiling before adapter overhead. QLoRA resolves this by quantizing the base model to 4-bit NF4 (~1.9 GB) and maintaining only the low-rank adapter weights in bf16. This is what makes 8 GB viable without compromising convergence.

4.2 Memory Optimisation — Decision Log

Every configuration choice was driven by explicit VRAM impact measurement, not rule-of-thumb defaults:

Component	Initial	Final	VRAM Impact
LoRA rank	32 across 7 modules	16 on q_proj, v_proj	−1.5 GB
Batch size	4	1 + grad accum ×8	−2.0 GB
Optimizer	AdamW	Adafactor	−1.0 GB
Gradient checkpointing	Enabled	Disabled (stable)	−1.0 GB
Compute dtype	fp16	bf16	Stable on Blackwell

4.3 Training Configuration

```
per_device_train_batch_size = 1
gradient_accumulation_steps = 8 # effective batch = 8
optim = "adafactor"
bf16 = True
learning_rate = 2e-4
num_train_epochs = 3
lora_r = 16
lora_target_modules = ["q_proj", "v_proj"]
```

4.4 Bugs Found and Fixed

SFTTrainer step-count explosion. Initial training with packing=True produced 2,139 steps on 90 samples — roughly 24× the expected count. Root cause: SFTTrainer with pre-tokenized input misinterprets packed sequence lengths, inflating the step count without processing more data. Fix: replaced SFTTrainer with a standard Trainer using explicit tokenization and DataCollatorForLanguageModeling. Step count normalised to the expected 2,673 steps across 3 epochs (891/epoch).

PyTorch 2.6 checkpoint incompatibility. Resuming from a checkpoint raised `UnpicklingError` due to the `weights_only=True` default change introduced in PyTorch 2.6. Fix: automated removal of `rng_state.pth` from checkpoint directories prior to resumption.

Both bugs would have silently corrupted training without careful step-count monitoring and checkpoint validation — not issues that appear in tutorials.

5. TRAINING RESULTS

5.1 Loss and Perplexity Progression

Epoch	Train Loss	Val Loss	Val Perplexity	Overfitting?
1	0.8511	0.9343	2.55	No
2	0.8367	0.8839	2.42	No
3	0.7780	0.8772	2.40 ✓	No

Validation loss decreased monotonically across all three epochs. The train-val gap remained narrow and stable — a sign that LoRA rank and dataset size were appropriately matched, not that the model was memorising. Final perplexity of 2.40 beats the target by 4×.

5.2 Domain Restriction in Practice

Query	Model Response
What is the capital of France?	I don't have enough information in my knowledge base to answer this.
Who won the FIFA World Cup?	I don't have enough information in my knowledge base to answer this.
How do I train a neural network in Python?	Refuses — then adds unsolicited general Python ML advice (see §6 Limitations).

5.3 Manual Evaluation (10-Sample Scoring)

Dimension	Score
Java correctness	8.5 / 10
Domain restriction	8.5 / 10
Training-data grounding	7.5 / 10
Hallucination control	8.0 / 10

6. LIMITATIONS & HONEST ASSESSMENT

The model has one systematic weakness worth flagging clearly: domain-adjacent technical queries (Python, general algorithms, ML) sometimes receive partial responses rather than clean refusals. The model starts to refuse, then continues with general knowledge — the boundary between the Java corpus and general programming knowledge leaks from the 3B's pretraining.

The other gap is strict corpus grounding. The model answers Java questions correctly but tends to draw on its pretraining knowledge rather than reproducing specific training corpus facts. This is a fundamental limitation of generative fine-tuning without retrieval — the model generalises, rather than retrieves.

Recommended next steps:

- **RAG layer** over the source corpus for strict factual grounding — the most impactful next step
- **DPO with curated refusal preference pairs** to sharpen the domain boundary on adjacent technical topics
- **Expanded unanswerable examples** targeting Python and general CS queries specifically

7. PRODUCTION DEPLOYMENT

LoRA adapters were merged into base model weights via `merge_and_unload()`, producing a single self-contained checkpoint with no adapter dependency at runtime. The merged model runs inference at under 2 GB VRAM — deployable on any GPU from the last five years, or on CPU for low-throughput use cases.

```
from transformers import AutoModelForCausalLM, AutoTokenizer
# Load directly from HuggingFace
model = AutoModelForCausalLM.from_pretrained("Debarun12/JavaExpert-Qwen2.5-3B")
tokenizer = AutoTokenizer.from_pretrained("Debarun12/JavaExpert-Qwen2.5-3B")
```

8. REPOSITORY STRUCTURE

```
.
├── data/
│   ├── java_corpus.txt # Raw 42k-line Java documentation corpus
│   ├── qa_pairs_chat.jsonl # 7,921 chat-formatted QA pairs
│   └── notebooks/
│       ├── QNA_dataset_generation.ipynb # Corpus preprocessing & QA generation
│       └── FineTuneQLoRA_final.ipynb # QLoRA fine-tuning pipeline
├── checkpoints/ # Per-epoch training checkpoints
├── adapters/ # Best LoRA adapter weights
└── merged_model/ # Production-merged model
```

9. KEY ENGINEERING CONTRIBUTIONS

Low-resource fine-tuning under hard constraints

Fine-tuned a 3B model on an 8 GB consumer GPU with <7 GB peak VRAM by combining QLoRA, Adafactor, and gradient accumulation — each chosen for a specific, measured memory impact rather than convention.

Data pipeline built for the problem

Cleaned a 42,000-line noisy PDF corpus, generated 7,921 QA pairs via an LLM-assisted loop with custom quality filtering, and structured the dataset with explicit negative examples for refusal training.

Debugging real training bugs, not tutorial problems

Found and fixed a silent SFTTrainer step-count inflation bug and a PyTorch 2.6 checkpoint incompatibility — issues that only surface when running real training jobs and watching the numbers carefully.

Evaluation-driven decisions

Used validation perplexity and loss curve monitoring to determine that 3 epochs was sufficient — preventing overfitting on a small dataset where a 4th epoch could have degraded generalisation.

Production-ready delivery

Merged adapters into the base model, verified inference VRAM, and published to HuggingFace — not a research artefact, a deployable system.

Model available at: <https://huggingface.co/Debarun12/JavaExpert-Qwen2.5-3B>

Debarun Das · June 2026 · NVIDIA GeForce RTX 5050 (8 GB VRAM)